

Entrega 3, FBD

CASO DE ESTUDIO – DANN ALPES

Diseño de la base de datos

a. Workload:

- a. Cuantifiquen las entidades (cantidad de registros que tendría la BD para cada una de las entidades, pueden encontrar un aproximado en el enunciado).

Reseñas: Debe haber una entidad que pueda guardar en la base de datos toda la información que requiere la reseña (es decir cada hotel tiene un arreglo de objetos tipo reseña), a continuación las características de cada uno de los objetos:

- fechaDeCreacion: string
- hotelDeControl: [nombreHotel: string, idHotel: string]
- horaDePublicado: date
- resenaRespectiva: string
- usuarioPublico: [nombre: string, email: string]
- nota: int (un valor del 1 al 5)
- público: booleano (Decir si esta disponible al público o no, automáticamente true pero cambia a false si el administrador lo decide)
- asunto: string
- utilidad: int
- respuestas: [respuesta: string, administrador: string]

Cientes: En la base de datos se guardará los usuarios que realizaron reservar, para permitir que se hizo la reserva correctamente, debió haber estado en el hotel:

- nombreUsuario: string
- contraseña: string
- email: string

Administrador: Al igual que el cliente, esté solo tendrá información de inicio de sesión, pero con distintos permisos a la hora de realizar la parte administrativa:

- nombreAdmin: string
- contraseñaAdmin: string
- emailAdmin: string

Hotel: Aquí esta la información general del hotel para poder tener saber a quién le pertenece cada reseña:

- nombreHotel: string
- administradorEnControl: [nombreAdmin: string, idAdmin: string]
- ubicacion: string
- ciudad: string

Cuantificación de cada dato:

Entidad	Cantidad estimada
Cientes	+50.000 registros
Reseñas	250.000 documentos en 2 años
Administradores	50–200 registros aprox
Hoteles	20–100 hoteles aprox
Votos Utilidad (cuántas operaciones ocurren por día en la app)	500 votos diarios

- b. Analicen las operaciones de lectura y escritura para cada entidad. Para ello utilicen una tabla como la del ejemplo del anexo A. Recuerden que este análisis sirve para saber qué información se accede de manera conjunta.

Entidad	Operación	Información Necesaria	Tipo
Administrador	Poder agregar un hotel a la base de datos	nombre del hotel + ubicación + ciudad	Write
Cliente + Reseña	El cliente tiene la posibilidad de crear una reseña a un hotel	reseña + fecha y hora actual + nota del 1 al 5 + asunto + usuario	Write
Administrador + Reseña	El administrador puede eliminar reseñas (no hacerla pública)	respectiva reseña	Write
Cliente + Reseña	El cliente tiene la posibilidad de editar una reseña que le corresponde	respectiva reseña + nuevo texto + usuario	Write
Cliente + Reseña	El cliente tiene la posibilidad de eliminar una reseña que le corresponde	respectiva reseña + usuario	Write
Cliente + Reseña	El cliente puede leer reseñas aunque no tenga cuneta	respectiva reseña	Read
Cliente + Reseña	El cliente puede votar sí una reseña le pareció útil	respectiva reseña + usuario	Write
Administrador + Reseña	El cliente tiene la posibilidad de responder una reseña que le corresponde	respectiva reseña + nuevo texto + usuario	Write
Administrador + Reseña	El administrador puede leer reseñas aunque no tenga cuneta	respectiva reseña	Read

Administrador + Reseña	El administrador tiene la posibilidad de eliminar una reseña que le corresponde	respectiva reseña + usuario de administrador	Write
Administrador + Reseña	El administrador tiene la posibilidad de destacar varias reseña	respectiva reseña + usuario de administrador	Write

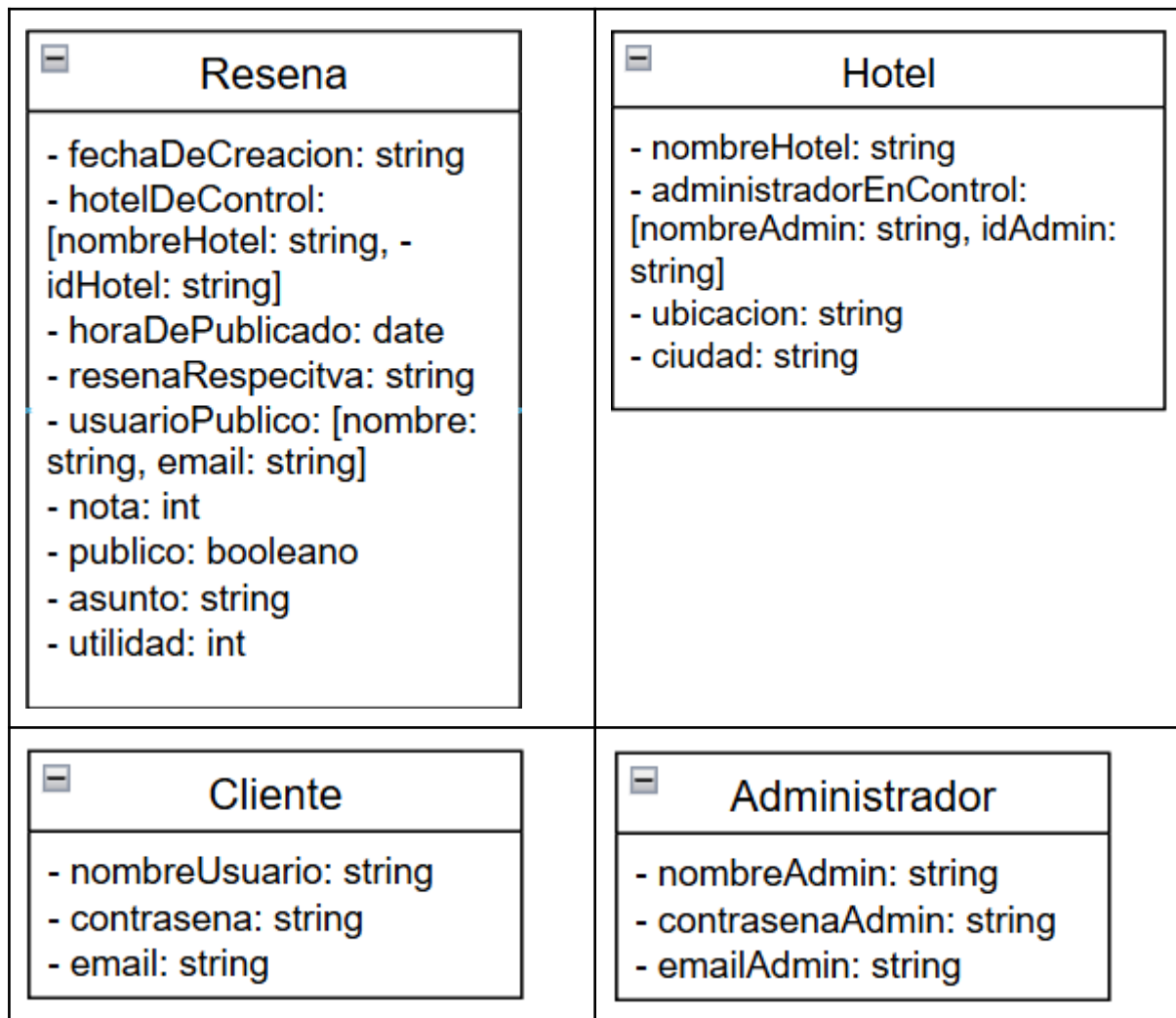
- c. Cuantifiquen las operaciones de lectura y escritura para cada entidad. Para ello utilicen una tabla como la del ejemplo del anexo B.

Entidad	Operación	Información Necesaria	Tipo	Cantidad estimada diaria
Administrador	Poder agregar un hotel a la base de datos	nombre del hotel + ubicación + ciudad	Write	1–5 operaciones/día
Cliente + Reseña	El cliente tiene la posibilidad de crear una reseña a un hotel	reseña + fecha y hora actual + nota del 1 al 5 + asunto + usuario	Write	300–500 operaciones/día
Administrador + Reseña	El administrador puede eliminar reseñas (no hacerla pública)	respectiva reseña	Write	20–50 operaciones/día
Cliente + Reseña	El cliente tiene la posibilidad de editar una reseña que le corresponde	respectiva reseña + nuevo texto + usuario	Write	100–200 operaciones/día
Cliente + Reseña	El cliente tiene la posibilidad de eliminar una reseña que le corresponde	respectiva reseña + usuario	Write	50–100 operaciones/día
Cliente + Reseña	El cliente puede leer reseñas aunque no tenga cuneta	respectiva reseña	Read	10.000 consultas/día
Cliente + Reseña	El cliente puede votar si una reseña le pareció útil	respectiva reseña + usuario	Write	500 operaciones/día
Administrador + Reseña	El cliente tiene la posibilidad de responder una reseña que le corresponde	respectiva reseña + nuevo texto + usuario	Write	200–300 operaciones/día
Administrador + Reseña	El administrador puede leer reseñas aunque no tenga cuneta	respectiva reseña	Read	500–1000 consultas/día

Administrador + Reseña	El administrador tiene la posibilidad de eliminar una reseña que le corresponde	respectiva reseña + usuario de administrador	Write	20–50 operaciones/día
Administrador + Reseña	El administrador tiene la posibilidad de destacar varias reseña	respectiva reseña + usuario de administrador	Write	5–20 operaciones/día

b. Modelo conceptual de las entidades:

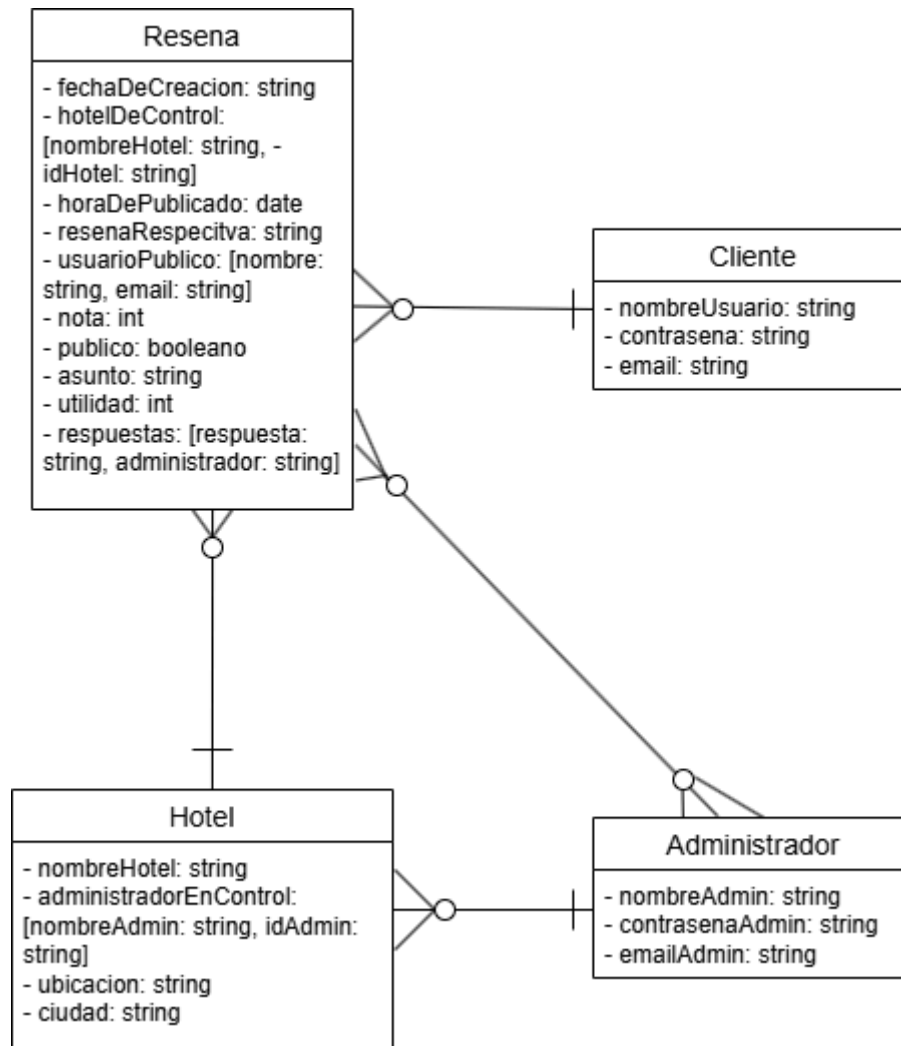
Estructura de cada una de las entidades:



Con base a esto podemos darle relación a las entidades:

- Cada cliente puede o no tener varias reseñas, pero si existe una reseña, esta debe siempre estar enlazada a un solo cliente.
- Si existe una reseña, debe estar enlazada a un hotel, y un hotel puede tener varias reseñas.
- Un administrador puede o no tener en control varios hoteles, si existe un hotel este debe de estar enlazado a un administrador.

- Una reseña puede o no tener varias respuestas de los administradores, y un administrador puede o no responder varias reseñas.

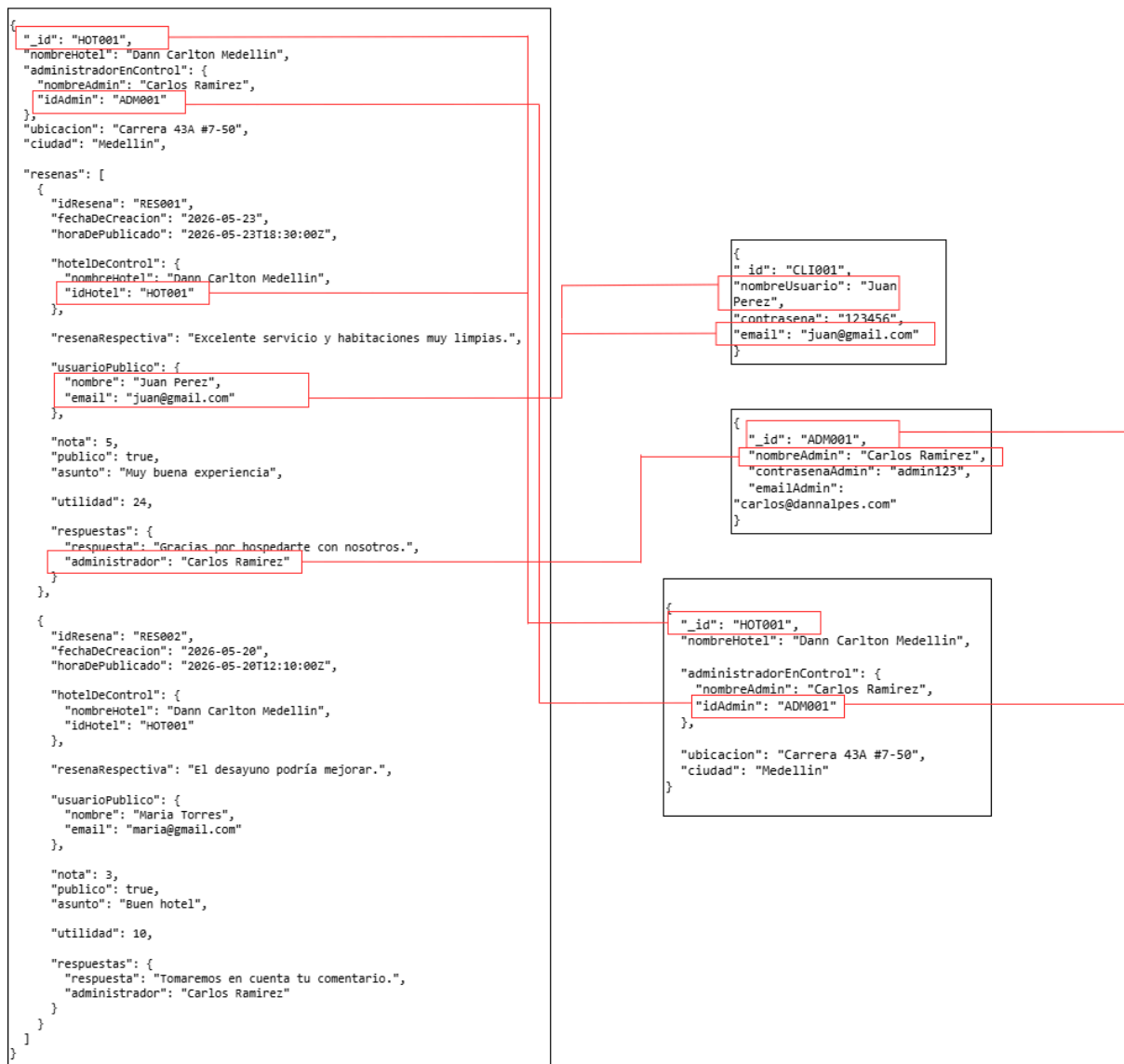


A continuación la tabla de análisis del esquema:

Relación entre entidades	Cardinalidad	Tipo de esquema	Justificación basada en workload
Hotel → Reseña	1:N	Embebido	Las reseñas se consultan frecuentemente junto con el hotel (10.000 consultas diarias), por lo que conviene mantener acceso conjunto rápido en MongoDB.
Cliente → Reseña	1:N	Referenciado	Un cliente puede crear

			múltiples reseñas y la información del cliente debe mantenerse independiente para evitar duplicación de datos.
Administrador → Hotel	1:N	Referenciado	Un administrador puede administrar varios hoteles y sus datos de autenticación deben centralizarse.
Administrador → Reseña (respuesta/moderación)	1:N	Referenciado	El administrador interactúa con múltiples reseñas mediante respuestas o moderación, pero la información administrativa no debe duplicarse en cada reseña.
Reseña → Respuesta Administrador	1:N	Embebido	La respuesta del administrador puede pertenecer a varios administradores, en el caso de haber varias respuestas.

A continuación la estructura JSON, para verlo con más detalle, por favor [ingresar a este link](#):



Explicación de las consultas:

Requerimiento de consulta 1:

```
@app.get("/consultas/top-hoteles")
def top_hoteles(
    fechaInicio: str = Query(..., description="Formato YYYY-MM-DD"),
    fechaFin: str = Query(..., description="Formato YYYY-MM-DD")
):
    pipeline = [
        { "$unwind": "$resenas" },
        { "$match": { "resenas.publico": True, "resenas.fechaDeCreacion": { "$gte": fechaInicio, "$lte": fechaFin } } },
        { "$group": { "_id": "$_id", "nombreHotel": { "$first": "$nombreHotel" }, "ciudad": { "$first": "$ciudad" },
            "calificacionPromedio": { "$avg": "$resenas.nota" }, "totalResenas": { "$sum": 1 } } },
        { "$sort": { "calificacionPromedio": -1 } },
```

```

    { "$limit": 10 },
    { "$project": { "_id": 0, "idHotel": "$_id", "nombreHotel": 1, "ciudad": 1,
                    "calificacionPromedio": { "$round": [ "$calificacionPromedio", 2 ] }, "totalResenas":
1 } }
  ]
return list(hoteles_col.aggregate(pipeline))

```

Explicación:

Esta consulta es la encargada de **obtener los 10 hoteles mejor calificados según las reseñas públicas dentro de un rango de fechas**. Para el funcionamiento se realizó las siguientes estrategias de consulta:

1. *Unwind*: Separa el arreglo de las reseñas para que cada reseña quede como un documento independiente, es decir, si un hotel tiene varias reseñas a su nombre, separa cada reseña por separado. Esto puede convertir el JSON en lo siguiente:

<pre> { "_id": 1, "nombreHotel": "Hotel Caribe", "resenas": [{ "nota": 5 }, { "nota": 4 }, { "nota": 3 }] } </pre>	<pre> { "_id": 1, "nombreHotel": "Hotel Caribe", "resenas": { "nota": 5 } } </pre>
	<pre> { "_id": 1, "nombreHotel": "Hotel Caribe", "resenas": { "nota": 4 } } </pre>
	<pre> { "_id": 1, "nombreHotel": "Hotel Caribe", "resenas": { "nota": 3 } } </pre>

La razón por este unwind es para poder evaluar cada reseña por separado.

2. *Match*: Tiene que cumplir que la reseña estuvo establecida en las fechas correspondientes. Igualmente, filtrar solo por aquellas reseñas que sean públicas.
3. *group*: Este es el encargado de agrupar de vuelta las reseñas que sean del mismo hotel, esto permite funcionar con el *avg*, que busca el promedio de las reseñas del mismo hotel.
4. *sum*: Este permite contabilizar cuantas reseñas tiene cada uno de los hoteles.
5. *first, sort, project y limit*: Cada uno da detalles para que cumpla la consulta exactamente como solicitada, esto hace respectivamente; **first**: Encuentra el nombre del hotel, por lo que solo es first de cada documento en lugar de preguntar varias veces; **sort**: Ordenar descendente para ver de mayor a menor; **project**: Mostrar sólo el nombre del hotel, el id del hotel, la ciudad, la clasificación promedio y el número de reseñas; **limit**: Mostrar solo los 10 primeros.

Requerimiento de consulta 2:


```

@app.get("/consultas/evolucion/{hotel_id}")
def evolucion_hotel(hotel_id: str, anio: int = Query(...)):
    hotel_exists(hotel_id)
    _id = parse_id(hotel_id)
    pipeline = [
        { "$match": { "_id": _id } },
        { "$unwind": "$resenas" },
        { "$match": { "resenas.publico": True } },
        { "$addFields": { "resenas.fechaDt": { "$toDate": "$resenas.horaDePublicado" } } },
        { "$group": { "_id": { "anio": { "$year": "$resenas.fechaDt" }, "mes": { "$month": "$resenas.fechaDt" } },
            "calificacionPromedio": { "$avg": "$resenas.nota" }, "totalResenas": { "$sum": 1 } } },
        { "$match": { "_id.anio": anio } },
        { "$sort": { "_id.mes": 1 } },
        { "$project": { "_id": 0, "anio": "$_id.anio", "mes": "$_id.mes",
            "calificacionPromedio": { "$round": [ "$calificacionPromedio", 2 ] }, "totalResenas":
1 } }
    ]
    return list(hoteles_col.aggregate(pipeline))

```

Explicación:

Esta consulta busca **mostrar la evolución mensual de las calificaciones de un hotel durante un año específico**. Para esto se realizó la siguiente estrategia de consulta:

1. *Match*: Este sirve para poder realizar la consulta con tan solo el hotel solicitado:
2. *Unwind*: Igual que el punto pasado, separa el arreglo de reseñas de un hotel a diferentes documentos con solo una reseña.
3. *Segundo match*: Filtra solo aquellas reseñas que son públicas.
4. *Addfields*: Esta permite la transformación del tipo de dato, convirtiendo las fechas en tipo Date en MongoDB para su análisis más claro. Luego se aplica el *toDate* para poder convertir la fecha en un objeto tipo fecha.
5. *group*: Agrupamos por año y por mes, esto para realizar otro match después para el año seleccionado.
6. *avg y sum*: Aquí calculamos la nota promedio de las reseñas de los grupos, y suma la cantidad de reseñas por cada mes del año seleccionado.
7. *tercer match*: Aquí filtramos por tan solo el año solicitado.
8. *sort y project*: Ordenamos por mes, es decir del mes 1 - 12, y luego proyectamos solo los datos solicitados por la consulta. Esto es: año, mes, clasificación promedio y número de reseñas.

Requerimiento de consulta 3:

```

@app.get("/consultas/comparativo")
def comparativo_ciudad(ciudad: str = Query(...)):
    pipeline = [
        { "$match": { "ciudad": str(ciudad) } },
        { "$addFields": { "resenas": { "$ifNull": [ "$resenas", [] ] } } },

```

```

    { "$addFields": {
      "resenaPublicas": { "$filter": { "input": "$resenas", "as": "r", "cond": { "$eq": [ "$$r.publico",
True] } } }
    } },
    { "$addFields": {
      "totalResenas": { "$size": "$resenaPublicas" },
      "calificacionPromedio": { "$cond": [ { "$gt": [ { "$size": "$resenaPublicas" }, 0 ] }, { "$avg":
"$resenaPublicas.nota" }, 0 ] },
      "resenasConRespuesta": { "$size": { "$filter": { "input": "$resenaPublicas", "as": "r",
"cond": { "$gt": [ { "$size": { "$ifNull": [ "$$r.respuestas", [] ] }, 0 ] } } } } },
      "resenasDestacadas": { "$size": { "$filter": { "input": "$resenaPublicas", "as": "r",
"cond": { "$eq": [ "$$r.destacada", True ] } } } } }
    } },
    { "$addFields": {
      "porcentajeConRespuesta": { "$cond": [ { "$gt": [ "$totalResenas", 0 ] },
      { "$round": [ { "$multiply": [ { "$divide": [ "$resenasConRespuesta", "$totalResenas" ] },
100 ] }, 1 ] }, 0 ] },
      "porcentajeDestacadas": { "$cond": [ { "$gt": [ "$totalResenas", 0 ] },
      { "$round": [ { "$multiply": [ { "$divide": [ "$resenasDestacadas", "$totalResenas" ] }, 100 ] },
1 ] }, 0 ] },
    } },
    { "$facet": {
      "hoteles": [
        { "$project": { "_id": 0, "idHotel": "$_id", "nombreHotel": 1,
          "calificacionPromedio": { "$round": [ "$calificacionPromedio", 2 ] },
          "totalResenas": 1, "porcentajeConRespuesta": 1, "porcentajeDestacadas": 1 } },
        { "$sort": { "calificacionPromedio": -1 } }
      ],
      "promedioDeciudad": [
        { "$group": { "_id": None, "promedio": { "$avg": "$calificacionPromedio" } } },
        { "$project": { "_id": 0, "promedio": { "$round": [ "$promedio", 2 ] } } }
      ]
    } }
  ]
}
resultado = list(hoteles_col.aggregate(pipeline))
if not resultado:
    return {"hoteles": [], "promedioDeciudad": 0}

data = resultado[0]
prom_ciudad = data["promedioDeciudad"][0]["promedio"] if data["promedioDeciudad"] else 0
for h in data["hoteles"]:
    h["bajoPromedio"] = h["calificacionPromedio"] < prom_ciudad

return {"ciudad": ciudad, "promedioDeciudad": prom_ciudad, "hoteles": data["hoteles"]}

```

Explicación:

La consulta realiza un análisis comparativo de los hoteles de una ciudad específica. A
 continuación la explicación de esta:

1. *Match*: Este sirve para filtrar únicamente los hoteles que pertenezcan a la ciudad solicitada en la consulta.

2. *Primer addFields:* Aquí verificamos que el campo reseñas exista. Si un hotel no tiene reseñas se reemplaza por un arreglo vacío usando `ifNull`. Esto evita errores en las siguientes operaciones.
3. *Segundo addFields:* Se crea un nuevo campo llamado `resenaPublicas`. Aquí se usa `filter` para quedarse únicamente con aquellas que sean públicas.
4. *Tercer addFields:* En esta etapa se calculan varias métricas importantes:
 - `totalResenas`: Usa `size` para contar cuántas reseñas públicas tiene el hotel.
 - `calificacionPromedio`: Usa `avg` para calcular el promedio de las notas de las reseñas públicas. Se utiliza para validar que sí existan reseñas; si no existen, el promedio será 0.
 - `resenasConRespuesta`: Filtra las reseñas públicas que tengan al menos una respuesta dentro del arreglo `respuestas`, y luego cuenta cuántas cumplen esa condición.
 - `resenasDestacadas`: Filtra las reseñas públicas que tengan el campo `destacada` en `True` y cuenta cuántas existen.
5. *Cuarto addFields:* Aquí se calculan porcentajes a partir de las métricas anteriores:
6. `porcentajeConRespuesta`: Divide las reseñas con respuesta entre el total de reseñas y multiplica por 100 para obtener el porcentaje.
Luego se usa `round` para redondear a un decimal.
`porcentajeDestacadas`: Calcula el porcentaje de reseñas destacadas respecto al total de reseñas. También se redondea a un decimal.
7. *Facet:* Esta etapa permite ejecutar dos consultas paralelas sobre los mismos datos.
 - `hoteles`:
 - `project`: Se seleccionan únicamente los campos requeridos para la respuesta final: `idHotel`, `nombreHotel`, promedio de calificación, total de reseñas y porcentajes calculados.
 - `sort`: Ordena los hoteles de mayor a menor según la calificación promedio.
 - `promedioDeCiudad`:
 - `group`: Agrupa todos los hoteles de la ciudad en un único grupo para calcular el promedio general de calificación de la ciudad usando `avg`.
 - `project`: Se devuelve únicamente el promedio redondeado a dos decimales.